

An Executable Benchmark for LLM-Based HLS Repair: Design Complexity and Repair Underconstraint

Maisha Mastora

Department of Electrical and Computer Engineering
University of New Hampshire
Durham, NH, USA
maisha.mastora@unh.edu

Dean Sullivan

Department of Electrical and Computer Engineering
University of New Hampshire
Durham, NH, USA
dean.sullivan@unh.edu

Abstract—Automated repair of High-Level Synthesis (HLS) designs using large language models (LLMs) is an emerging but underexplored problem. While LLM-based repair has shown strong results on register-transfer level (RTL) Verilog, the only prior systematic study of HLS logic repair reports just 10.5% correction accuracy for GPT-4—with no analysis of why repair fails or what drives difficulty. In this paper, we present the first comprehensive evaluation of LLM-based HLS repair across four models (GPT-4o, GPT-4o-mini, GPT-5.4, and Claude Opus 4.6), evaluated on 125 benchmark instances spanning eight logic bug types across three open-source HLS suites (CHStone, MachSuite, and Polybench). We construct, to the best of our knowledge, the first executable APR-style HLS repair benchmark with suite-specific functional oracles, enabling pass@k evaluation rather than the string-match approximations used in prior work. Our results reveal that design context and scale, rather than bug type alone, dominate repair difficulty: repair rates range from 6–45% on complex cryptographic kernels (CHStone) to 81–93% on compact algorithmic kernels (MachSuite) despite identical bug type distributions. We further introduce *solution multiplicity*—the fraction of distinct patches generated across repeated repair attempts—as an empirical measure of repair underconstraint, and show that it strongly predicts repair failure across all four models (Spearman $\rho = -0.393$, $p = 6.61 \times 10^{-20}$). Frontier models reduce underconstrained instances from 41–46 (GPT-4o, GPT-4o-mini) to just 3 (Claude Opus 4.6), with overall pass@1 improving from 42–53% to 73–76%. SHFT bugs remain consistently hard across all models, and semantic bug classes such as buffer indexing become reliably repairable only at frontier model scale. These findings establish a principled framework for understanding LLM repair limitations in HLS and show that future APR benchmarks must include complex, executable, and weakly identifiable designs that remain challenging after frontier LLM repair.

Index Terms—automated hardware repair, high-level synthesis, large language models, repair identifiability, solution multiplicity, program repair benchmarks

I. INTRODUCTION

Automated repair of hardware designs is an active research problem with practical stakes: logic bugs discovered after tape-out cost orders of magnitude more to fix than bugs caught during design. The manual hardware design debugging cycle consists of reproduce, localize, patch, and re-verify. This

process consumes a substantial fraction of front-end engineering time. For designs constructed in Verilog at the register-transfer level (RTL), search-based tools such as CirFix [1] and symbolic tools such as RTL-Repair [2] demonstrate correct repair on standard benchmarks, and LLM-based approaches now achieve pass@1 rates above 50% [3], [4] with continued improvement at each model generation.

High-level synthesis (HLS) raises the design abstraction level, allowing hardware to be specified in C/C++ and compiled to RTL by a synthesis tool. HLS is increasingly common in FPGA and ASIC workflows [5], and it broadens the hardware development pathway with more flexibility. Automated debugging support at this level raises practical importance. Yet LLM-based repair at the HLS level is barely understood. The only prior systematic study, HLSDebugger [6], reports that GPT-4 achieves just 10.5% correction accuracy on HLS logic bugs under zero-shot evaluation. No prior work explains *why* repair is so difficult at this level, or what properties of a design instance drive repair success or failure.

A second motivation is benchmark adequacy. If modern LLMs already repair most instances in a benchmark, then that benchmark is no longer a useful stress test for sophisticated APR engines. This concern is especially relevant for HLS, where many public benchmarks are compact kernels with short source files and limited functional oracles. We therefore study not only whether LLMs can repair HLS bugs, but whether existing HLS benchmark families remain difficult enough to evaluate next-generation repair engines.

This paper addresses that gap with three contributions.

First, we construct, to the best of our knowledge, the first executable APR-style HLS repair benchmark with functional oracles. We build 125 validated instances spanning eight C-logic bug types across three open-source HLS suites (CHStone, MachSuite, Polybench), pairing each buggy design with a functional oracle derived from the suite’s native testing infrastructure. Prior work [6] relies on string matching against ground truth snippets; our benchmark enables pass@k evaluation, which is standard practice in software repair and avoids conflating plausible-but-incorrect patches with correct ones.

Second, we evaluate four LLMs—GPT-4o, GPT-4o-mini, GPT-5.4, and Claude Opus 4.6—on all 125 instances using a uniform zero-shot prompt and report pass@1, pass@5, and per-bug-type breakdowns. Our results reveal that not only the bug type, but also design context and scale, significantly determine repair difficulty. Pass@1 ranges from 6–45% on CHStone’s large cryptographic kernels to 81–93% on MachSuite’s compact algorithmic kernels, despite both suites containing identical bug type distributions drawn from the same source dataset. This finding holds across all four models and implies that existing single-suite evaluations significantly mischaracterize LLM repair capability.

Third, we introduce *solution multiplicity*—the fraction of distinct patches across N repair attempts—as an empirical measure of repair underconstraint. High multiplicity on failing instances indicates the LLM is exploring without converging on a correct fix, reflecting insufficient observational constraint rather than model incapability. We show that solution multiplicity is strongly negatively correlated with repair success across all four models (Spearman $\rho = -0.393$, $p = 6.61 \times 10^{-20}$), and that frontier models nearly eliminate underconstrained instances: from 41–46 such instances for GPT-4o and GPT-4o-mini, down to 3 for Claude Opus 4.6.

The rest of the paper is organized as follows. Section II reviews related work. Section III motivates why HLS repair is an identifiability challenge. Section IV describes benchmark construction and experimental setup. Section V presents results. Section VI discusses implications, and Section VII concludes.

II. BACKGROUND AND RELATED WORK

A. Automated RTL Repair

Automated repair for hardware designs has been focused on RTL verilog variants. Early search-based approaches such as CirFix [1] applied genetic programming over a mutation operator space to repair functional bugs in Verilog designs. Subsequent work explored symbolic methods: RTL-Repair [2] formulates repair as a constraint satisfaction problem over candidate patch expressions, achieving substantially faster repair times on datapath-dominated designs. SRepair [7] extends symbolic repair through regression-based patch ranking. [8] presents a dictionary-guided HDL repair system that combines Another Tool for Language Recognition (ANTLR)-derived design specific mutation vocabularies with a simulation-divergence fault localization (FL) for RTL program repair.

More recently, LLMs have been applied directly to RTL repair. RTLFixer [9] demonstrated that LLMs can resolve RTL syntax errors with high reliability. HDLDebugger [10] extended this to functional debugging through retrieval-augmented generation. LiK [4] showed that accurate bug localization improves repair substantially, with functional pass@1 rising from 40.39% to 58.92% on Verilog benchmarks. ARSP [3] introduced semantic partitioning to guide LLM repair, reporting pass@1 of 77.92% and pass@5 of 83.88%. CraftRTL [11] demonstrated that frontier models already achieve competitive performance on Verilog repair

without fine-tuning, and VeriRAG [12] showed that retrieval augmentation yields up to a 7.72-fold improvement over zero-shot baselines.

Collectively, these results establish LLM-based RTL repair as a mature and rapidly improving area. Critically, all of these efforts operate exclusively at the RTL abstraction level. No prior work has systematically evaluated LLM repair effectiveness on HLS logic bugs with functional oracles. HWE-Bench [13] confirms that existing hardware repair benchmarks contain no HLS instances. This gap motivates the present work.

B. LLM-Based HLS Debugging and Repair

High-level synthesis enables hardware generation from C/C++ descriptions, broadening development to engineers without RTL expertise [5]. Debugging HLS designs presents distinct challenges: the semantic gap between C++ source behavior and synthesized hardware behavior means bugs can be non-obvious at the source level [14].

Existing work on LLM-based HLS assistance has primarily targeted code generation rather than repair. HLSPilot [5] explores feedback-driven generation with synthesis constraints, while SynthAI [15] automates pragma insertion and design refinement. For HLS repair, Xu et al. [16] proposed repairing C/C++ for HLS synthesizability, targeting unsupported constructs such as recursion and dynamic memory.

The most directly relevant prior work is HLSDebugger [6], which targets logic bug identification and correction in HLS C++ code. HLSDebugger introduces a labeled dataset of 300K HLS bug instances spanning eight logic bug types and proposes a fine-tuned encoder-decoder architecture for simultaneous bug localization and correction. Under zero-shot evaluation, GPT-4 achieves only 10.5% correction accuracy on the Chrysalis benchmark [14], [17], while their fine-tuned model reaches 37.6%. Crucially, HLSDebugger evaluates correction through string matching against ground truth snippets rather than functional testbench execution, and does not investigate what structural properties drive repair difficulty. Our work is complementary: we replace direct string-match scoring with executable oracles and shift the question from aggregate accuracy to what makes a design hard.

III. MOTIVATION

A. Repair Identifiability and Underconstraint

The distinction between *plausible* and *correct* repairs is well established in software APR [1]: a plausible repair passes all available tests but may not be semantically correct, a failure mode rooted in test suite incompleteness. In the hardware domain, the same mechanism applies: a weak oracle admits multiple structurally distinct patches, making the repair problem *underconstrained*. Hardware repair benchmarks typically characterize instances only by bug type, without accounting for how strongly the oracle constrains the repair search. This work addresses that gap for HLS by introducing *solution multiplicity* as an empirical metric of identifiability computable directly from LLM repair behavior.

B. Repair Identifiability in HLS

A repair instance is *identifiable* if the available test observations uniquely constrain the correct patch, and *underconstrained* if multiple structurally distinct patches all satisfy the oracle. While this distinction is implicit in software APR [1], hardware repair benchmarks have not explicitly characterized instances along this dimension. We identify three structural properties of HLS designs that make underconstraint particularly prevalent, and that explain why single-suite evaluations produce misleading repair rate estimates.

Single-vector oracles. HLS benchmarks typically contain one fixed test input. CHStone’s AES program exercises only the 128-bit key path; bugs injected into the 192 or 256-bit key paths pass the oracle silently. This coverage gap was the primary rejection criterion during benchmark construction: 75 candidate instances were discarded because the oracle could not distinguish buggy from correct behavior. Single-vector coverage is specific to HLS benchmark suites and makes underconstraint a structural property of the evaluation, not an edge case.

Algorithmic opacity. HLS source is a C function body with no explicit timing, wire assignments, or signal topology. An LLM repairing an HLS bug must infer hardware-semantic intent from C source alone. For complex designs such as AES key scheduling or GSM channel coding, many syntactically valid edits satisfy the oracle while implementing incorrect hardware behavior, increasing the plausible-but-incorrect patch space.

Context dilution from entry-point compilation. HLS tools compile from a single entry file that `#include` all helper modules. CHStone’s AES benchmark aggregates ~ 40 KB of source into one context; the bug site is a few lines within that. Prior work identifies bug signal dilution as a key LLM failure mode in long-context repair [3]. HLS amplifies this problem because entry-point compilation is a toolchain requirement, not a design choice. Our results confirm the effect directly. CHStone instances (~ 40 KB) achieve 6–45% pass@1 versus 81–93% for MachSuite (~ 3 KB), despite identical bug type distributions.

These three properties compound rather than act independently. The large entry-point programs are also the cryptographic kernels hardest to infer from source and the ones whose single-vector harnesses leave the widest margin for plausible-but-wrong patches, which is why CHStone has the hard cases. We observe the combined effect empirically through *solution multiplicity* (defined in Section IV), which measures patch diversity across repeated repair attempts without requiring formal specification analysis or ground-truth models.

IV. EXPERIMENTAL SETUP

Figure 1 summarizes the full workflow. The three HLS suites serve a dual role: they are the origin of the Chrysalis bug specifications and they supply the executable programs and oracles into which those bugs are injected. The injection pipeline produces 125 validated instances, which are repaired

by four LLMs and evaluated through suite-specific functional oracles before metric computation and analysis.

A. HLS Benchmark Construction

We build an executable HLS repair benchmark by pairing bug specifications from Chrysalis-HLS [14] with three open-source HLS suites: CHStone [18], MachSuite [19], and Polybench [20]. Chrysalis provides 1,500+ function-level bug specifications across 18 bug types without oracle or testbench, only for string-match evaluation [6] and cannot verify functional correctness of a repair; we restrict to the eight C-logic-compatible types testable with `gcc`: out-of-bounds array access (OOB), bit shift errors (SHFT), infinite loops (INF), manual loop unrolling errors (MLU), wrong zero-initialization (ZERO), unintended sign extension (USE), buffer indexing errors (BUF), and uninitialized variable reads (INIT). We filter to single-bug instances only (Chrysalis Error Size = 1).

For each candidate instance, our injection pipeline: (1) copies the benchmark source to a `correct/` directory and verifies it passes the oracle; (2) injects the Chrysalis faulty code snippet via string replace into a `buggy/` copy; (3) verifies the buggy version fails the oracle. Instances that fail verification in either direction are discarded. This two-sided check makes the benchmark executable rather than declarative: any patch that flips the oracle back to passing is a genuine functional repair under the suite’s own test.

CHStone (41 instances). Each CHStone program has a self-checking `main()` that sets `main_result = 0` on correct output. Oracle: `gcc -O0 <entry>.c -lm; pass iff exit code = 0`. INF bugs are detected by a 10-second timeout. JPEG was excluded due to duplicate-symbol compilation issues. Five programs contributed instances: AES (17), ADPCM (16), GSM (5), DFADD (2), Blowfish (1).

MachSuite (40 instances). MachSuite provides a harness-based oracle (`check_data()` in `local_support.c`) that compares output to a reference `check.data` file and returns exit code `-1` on failure. Compilation: `gcc -O0 -I<common> common/support.c common/harness.c local_support.c <kernel>.c -lm`. 10 out of 14 kernels compile cleanly; four were skipped due to compilation issues leading to oracle failures.

Polybench (44 instances). Polybench has no built-in self-check. We implement a differential oracle: compile the correct version with `-DPOLYBENCH_DUMP_ARRAYS`, capture `stderr` as `ref_output.txt`, then compare patched-version output byte-for-byte against reference. Reference output is generated once per kernel and reused across all instances. All 30 kernels compile and produce deterministic output; 27 have matching Chrysalis bug specifications.

The final benchmark contains 125 instances across eight bug types (Table I). BUF and INIT are underrepresented because their bug sites tend to fall in code paths not covered by single test vectors.

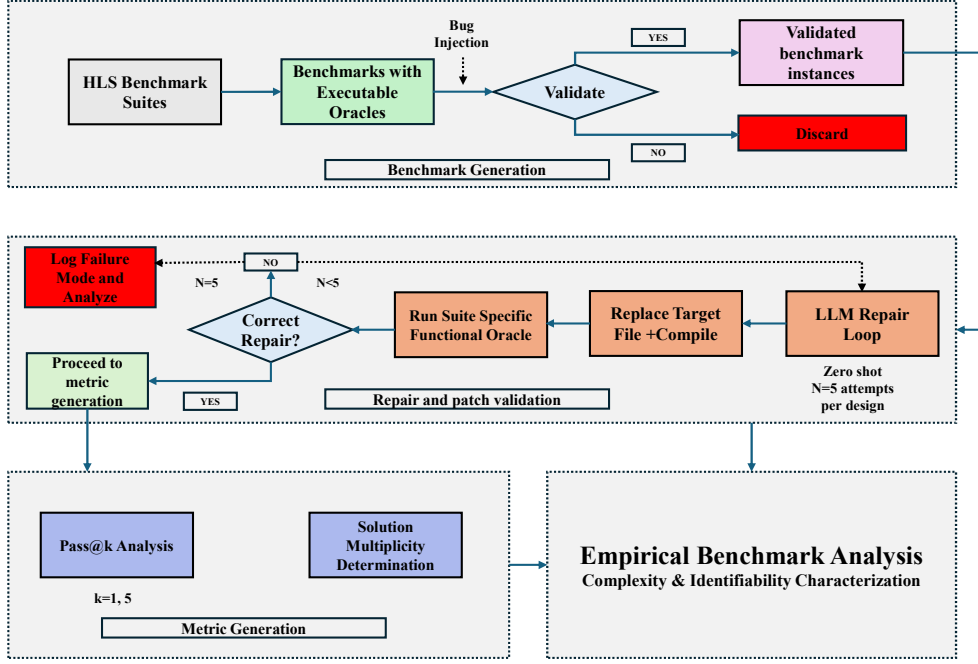


Fig. 1. System overview. HLS benchmark suites are paired with executable oracles and validated via bug injection to produce a benchmark of correct and buggy instance pairs. Validated instances are repaired zero-shot by LLMs ($N=5$ attempts per design); each candidate patch is recompiled and checked against a suite-specific functional oracle, then scored by pass@ k and solution multiplicity. Failure modes and repair outcomes are aggregated for empirical complexity and identifiability characterization.

TABLE I
BUG TYPE DISTRIBUTION ACROSS 125 BENCHMARK INSTANCES.

Bug Type	CHStone	MachSuite	Polybench	Total
MLU	7	10	9	26
ZERO	7	7	10	24
INF	7	5	10	22
OOB	7	8	3	18
USE	4	3	10	17
SHFT	7	1	0	8
INIT	1	5	1	7
BUF	1	1	1	3
Total	41	40	44	125

B. LLM Repair Pipeline

All models receive a uniform zero-shot prompt: “The following C source file contains bugs. Fix all of them and return the complete corrected file.” No hints about bug location or type are provided. The full buggy source file is passed as the user message; the model returns a complete replacement file. We run $N = 5$ independent repair attempts per instance at temperature 1.0. Token budgets use a per-instance dynamic allocation (min 1024, max 16384 tokens) based on source file length. We report model names as used in our experiment logs; exact API endpoint identifiers and evaluation dates are included in the artifact metadata, which will be released for public availability upon the acceptance of the paper.

We evaluate four models: **GPT-4o** (OpenAI, standard generation), **GPT-4o-mini** (OpenAI, cost-reduced), **GPT-5.4** (Ope-

nAI Batch API), and **Claude Opus 4.6** (Anthropic Batch API). GPT-4o and GPT-4o-mini represent the generation-4 frontier; GPT-5.4 and Opus 4.6 represent the current generation-5 frontier. The prompt is deliberately minimal to observe repair identifiability—whether the oracle and source together constrain the patch without any external information, so a fixed zero-shot prompt isolates instance and model-level variation. This establishes the baseline on which guided methods can later be evaluated.

C. Metrics

pass@ k uses the standard unbiased estimator from Chen et al. [21]:

$$\text{pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \quad (1)$$

where $n = 5$ is the total runs per instance and c is the number of passing runs. pass@1 is the primary metric; pass@5 gives an upper bound on what is achievable with a fixed sample budget.

Solution multiplicity is defined as the number of distinct patches divided by the total number of runs ($n = 5$). Two patches are distinct if they differ in any character after stripping whitespace. Multiplicity = 1.0 means every run produced a different patch; multiplicity < 0.5 indicates convergence on a small set of repairs. Because multiplicity is model- and decoding-dependent, we interpret it as an empirical repair-

TABLE II
OVERALL PASS@1, PASS@5, AND SOLUTION MULTIPLICITY BY MODEL.

Model	pass@1	pass@5	Multiplicity
GPT-4o-mini	42.4%	63.2%	0.970
GPT-4o	53.3%	70.4%	0.954
GPT-5.4	73.4%	78.4%	0.771
Claude Opus 4.6	75.7%	78.4%	0.310

TABLE III
PASS@1 BY SUITE AND MODEL. THE CHSTONE VS. MACHSUITE GAP HOLDS ACROSS ALL MODELS DESPITE IDENTICAL BUG TYPE DISTRIBUTIONS.

Suite	GPT-4o-mini	GPT-4o	GPT-5.4	Opus 4.6
CHStone	21.0%	13.7%	37.6%	44.9%
MachSuite	48.5%	74.0%	92.5%	90.5%
Polybench	56.8%	71.4%	89.5%	90.9%

behavior metric rather than an intrinsic property of the benchmark instance alone.

Failure mode categorizes each non-passing patch as: compile failure (patch does not compile), test failure (compiles but oracle fails), or timeout (INF-type bugs that do not terminate).

V. RESULTS

A. Overall Repair Performance

Table II reports pass@1 and pass@5 for all four models across all 125 instances. There is a clear generational progression: GPT-4o-mini (42.4% pass@1) < GPT-4o (53.3%) < GPT-5.4 (73.4%) $\approx$ Opus 4.6 (75.7%). This 33-percentage-point spread confirms that model capability significantly influences HLS repair success, consistent with findings in RTL repair [3], [4].

The reduction in multiplicity is as important as the increase in pass rate. Weaker models generate highly diverse patches across attempts, while Opus 4.6 often converges to the same patch. This indicates that stronger models are not only more accurate but also more stable in the repair space.

B. Design Complexity Dominates Repair Difficulty

RQ1: Does design complexity predict repair difficulty more strongly than bug type?

Table III breaks pass@1 down by suite. The suite-level variation is dramatic and consistent across all models: CHStone achieves 13.7–44.9%, while MachSuite achieves 48.5–92.5% and Polybench 56.8–90.9%. Critically, all three suites draw bugs from the same Chrysalis specifications and contain the same eight bug types—the only structural difference is design complexity and file size (~ 40 KB for CHStone vs. ~ 3 –4 KB for MachSuite/Polybench).

The MachSuite–CHStone gap ranges from 27.5 percentage points (Opus 4.6) to 60.5 points (GPT-4o), answering RQ1 affirmatively: suite complexity dominates bug type as a repair difficulty predictor. Because the bug-type mix is held fixed across suites by construction, this gap cannot be attributed to one suite containing intrinsically harder bug categories. MachSuite and Polybench are essentially solved by frontier models ($>90\%$); CHStone remains hard for all models.

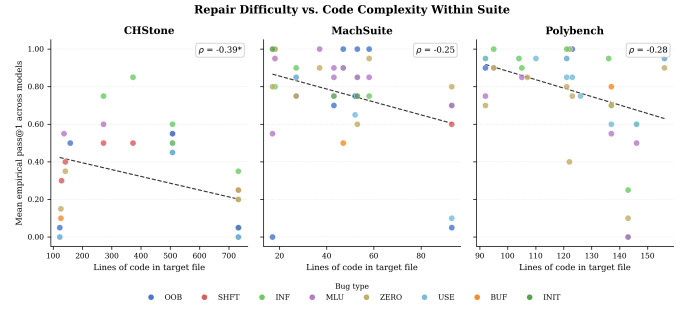


Fig. 2. Repair difficulty versus target-file LOC within each suite. Each point is one benchmark instance, colored by bug type. The y-axis reports mean empirical pass@1 across models. CHStone shows a significant negative correlation ($\rho = -0.389$, $p = 0.012$); MachSuite and Polybench show weaker non-significant negative trends due to narrower LOC ranges.

Figure 2 further shows that complexity is not only a between-suite effect. Within CHStone, target-file LOC is significantly negatively correlated with empirical pass@1 ($\rho = -0.389$, $p = 0.012$), confirming that repair difficulty scales continuously with file size even within a single suite. In MachSuite and Polybench, where file sizes are more uniform, within-suite correlations do not reach significance, consistent with range restriction rather than absence of effect.

C. Solution Multiplicity and Underconstraint

RQ2: Does solution multiplicity predict repair failure?

Solution multiplicity is strongly negatively correlated with pass@1 across all 125 instances (Spearman $\rho = -0.393$, $p = 6.61 \times 10^{-20}$). Instances where the model produces a different patch on every run are overwhelmingly failures; instances where the model converges on one or two patches are overwhelmingly passes. The correlation is computed over the full dataset, so it does not depend on any single suite or bug category.

Model-level multiplicity tracks capability inversely: GPT-4o-mini (0.970) > GPT-4o (0.954) > GPT-5.4 (0.771) > Opus 4.6 (0.310). Opus 4.6’s multiplicity of 0.310 is the lowest of any model: it produces the same correct patch on all 5 runs for instances it can repair, and similarly converges (incorrectly) on failing instances, reflecting high model confidence.

We define an *underconstrained* instance as one where multiplicity ≥ 0.8 and pass@5 = 0: the model generates maximally diverse patches and none pass. Table IV shows the count by model and suite. Older models have 35–46 such instances, concentrated on CHStone; frontier models reduce this to 3 (Opus 4.6). The 3 remaining underconstrained instances for Opus 4.6 involve large cryptographic files or complex loop structures where the oracle covers only a subset of code paths.

Multiplicity also separates two failure modes that a pass-rate-only view would merge. On the 3 BUF instances, GPT-4o records multiplicity 0.800 with 0.0% pass@1: it produces similar wrong patches every run, a confident-but-incorrect convergence. SHFT shows the opposite signature (multiplicity 1.000, 5.0% pass@1 for GPT-4o): a different guess each run, the hallmark of pure underconstraint. The same zero pass rate

TABLE IV
UNDERCONSTRAINED INSTANCES (MULTIPLICITY ≥ 0.8 , PASS@5 = 0)
BY MODEL AND SUITE.

Model	CHStone	MachSuite	Polybench	Total
GPT-4o-mini	23	12	10	46
GPT-4o	25	5	5	35
GPT-5.4	20	0	3	23
Opus 4.6	2	1	0	3

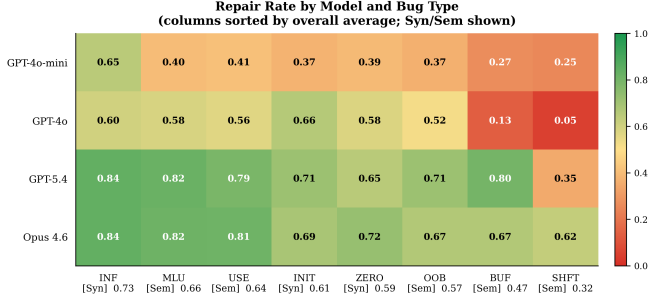


Fig. 3. Repair rate by model and bug type. Columns sorted by overall average empirical pass@1. Model scaling improves most bug types, but SHFT remains the hardest category. BUF shows strong improvement in larger models, suggesting some semantic bug classes become repairable only at frontier scale.

thus arises from two distinct causes, and multiplicity is what tells them apart.

D. Bug-Category Analysis

RQ3: Which bug types are consistently hard, and which improve with model scale?

Figure 3 shows empirical pass@1 by model and bug type. Model scaling improves most bug types, but SHFT remains the hardest category and buffer-indexing bugs improve only with frontier models.

Four patterns are notable. (1) **INF is consistently easiest**: loop termination conditions are syntactically localized and semantically obvious across all suites and models. (2) **SHFT is consistently hardest**: correct shift amounts require bit-level precision the model rarely recovers without localization hints; even Opus 4.6 achieves only 62.5%, and SHFT carries enough instances ($n = 8$) for this to be a stable trend. (3) Among the smaller categories, **BUF ($n = 3$) illustrates a semantic capability threshold**: GPT-4o repairs none of its instances while the generation-5 models repair most, consistent with buffer-indexing semantics being learnable but frontier-dependent; we read this as illustrative rather than a precise rate given the category size. (4) **MLU is suite-dependent**: GPT-4o achieves 2.9% on CHStone MLU but 55.4% overall, because MachSuite and Polybench MLU instances are in small files where the loop structure is clear. The aggregate and suite-level conclusions rest on the full dataset and do not depend on the smallest per-type cells.

Figure 4 shows empirical pass@1 by bug type and suite, illustrating how the same bug type can carry very different repair rates depending on design context.

pass@k gain by bug type. SHFT benefits most from additional attempts (+15–37% from pass@1 to pass@5),

Repair Rate by Bug Type and Suite
(sorted by overall average; Syn/Sem category shown)

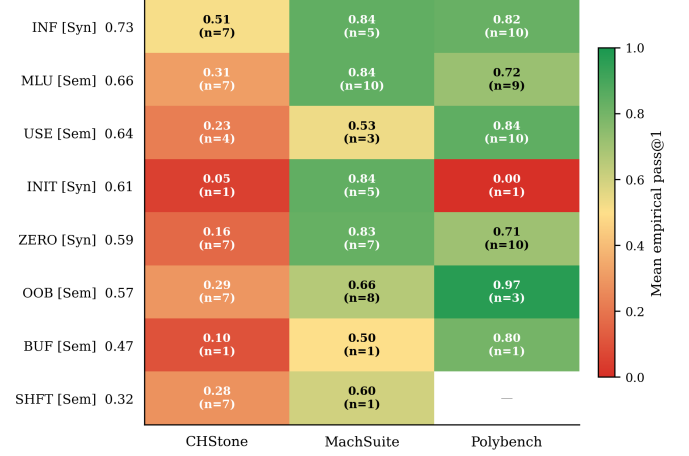


Fig. 4. Repair rate by bug type and suite. Each cell reports mean empirical pass@1 across models, with the number of instances in parentheses. The same bug type can have very different repair rates across suites, confirming that benchmark context substantially mediates bug-type difficulty.

confirming the correct shift value can occasionally be sampled but is not reliably produced. USE plateaus early (+3.5% for GPT-4o), indicating the model either knows the sign-extension fix or does not. BUF at 0.0% for GPT-4o shows zero gain at any k —not a sampling problem but a semantic capability gap that only larger models overcome.

E. Structural Metrics and Solution Multiplicity

To compare alternative predictors of repair difficulty, we analyze target-file LOC, cyclomatic complexity (CCN) [22], and solution multiplicity. Following established practice for nonparametric evaluation in SE research [23], we use Spearman rank correlation (ρ) throughout.

LOC is the strongest predictor ($\rho = -0.564$, $p < 0.001$), primarily reflecting between-suite scale differences. Cyclomatic complexity yields a non-significant positive correlation ($\rho = +0.174$, $p = 0.052$), the *opposite* of the structural complexity hypothesis. CCN is near-uniform across suites (avg. 6.95, 6.92, and 6.36 for CHStone, MachSuite, and Polybench respectively), confirming this is not a suite-level confound: CHStone files are large but not structurally more branchy. This is consistent with recent findings that classical metrics show no consistent correlation with LLM difficulty after controlling for length [24]. Solution multiplicity achieves $\rho = -0.393$ ($p < 0.001$), capturing repair difficulty that CCN cannot. LOC and multiplicity thus capture complementary dimensions: structural scale and semantic underconstraint.

Syntactic vs. semantic bug categories. We classify INF, INIT, and ZERO as syntactic and SHFT, BUF, OOB, MLU, and USE as semantic bugs. For weaker models a visible gap exists (GPT-4o-mini: 0.47 vs. 0.34; GPT-4o: 0.61 vs. 0.37), but it narrows sharply for frontier models (GPT-5.4: 0.73 vs. 0.69; Opus 4.6: 0.75 vs. 0.72). A Mann-Whitney U test does not reach significance ($p = 0.159$). The syntactic/semantic distinction describes a capability threshold that frontier models

have largely overcome [25], [26] rather than a complexity-independent predictor. Figure 5 summarizes the three predictor correlations.

VI. ANALYSIS AND DISCUSSION

Benchmark saturation and the need for harder APR tasks. Our results make a strong methodological argument: reporting repair rates on a single HLS suite produces misleading estimates of LLM capability and may overstate progress in APR. The CHStone-only repair rate for GPT-4o (13.7–25.9%) and the MachSuite rate (74–92.5%) would support opposite conclusions about HLS repair difficulty. More importantly, MachSuite and Polybench are largely saturated by frontier models, with repair rates above 90% in several settings. If an LLM can already solve most instances in such suites under a zero-shot prompt, then these suites alone cannot serve as strong benchmarks for more sophisticated APR engines. The true picture is that difficulty is driven by design complexity—file size, algorithmic depth, and oracle coverage—which varies more within HLS benchmarks than across bug types. A useful HLS APR benchmark should therefore not be defined by bug-type coverage alone. It should include executable functional oracles rather than string matching, span both compact kernels and larger entry-point programs, retain only oracle-observable bugs, include instances that remain hard for frontier LLMs, and report metadata such as LOC, oracle structure, failure mode, and identifiability metrics such as solution multiplicity.

Construction discipline and evaluation criterion. The 125-instance suite was built under a strict open-source, no-proprietary-tool constraint. Candidate Chrysalis-HLS specifications were retained only when the original program passed its oracle, the injected program compiled, and the injected bug was observable; the rest were rejected. The suite is therefore a systematically filtered executable subset whose every member carries a guaranteed pass/fail-discriminating test, which is what licenses the pass@k semantics used throughout. We score repair at the C level using each suite’s executable oracle—the standard correctness criterion in software APR—so the object of study is C-level functional repair: producing a source patch that restores externally observable behavior. This is the level at which HLS bugs are written, at which Chrysalis specifications are defined, and at which a repair must first be correct, making it the natural foundation for later work

that layers post-synthesis equivalence checks onto the same harness.

Solution multiplicity as an LLM-native metric. Standard metrics (pass@k, compile rate, test fail rate) measure outcomes. Solution multiplicity measures the repair process: is the model converging on an answer, or exploring randomly? Our correlation results ($\rho = -0.393$) show that multiplicity is a strong early signal of repair difficulty that is practically useful without ground-truth labels. Given a new benchmark instance, running 5 repair attempts and computing multiplicity provides an immediate estimate of whether the instance is identifiable by the model. High multiplicity (≥ 0.8) with no passing runs is a reliable flag for oracle coverage gaps or context length problems.

Failure mode analysis. Compile failures are stable at 11–20% regardless of suite or bug type, reflecting consistent LLM hallucination of invalid C syntax. Test failures (plausible-but-incorrect patches) are complexity-dependent: 54.1% on CHStone vs. much lower on simple kernels. This aligns with software APR findings that test suite coverage drives plausible-repair prevalence: CHStone’s single hardcoded test vector accepts a wide range of wrong patches.

Frontier model saturation. GPT-5.4 and Opus 4.6 are tied at pass@5 (78.4%) and both saturate MachSuite/Polybench above 90%. The marginal gain from pass@1 to pass@5 shrinks for frontier models, indicating that they either know the fix early or consistently fail. CHStone remains unsaturated at 51.2% pass@5 for Opus 4.6, making it the relevant hard benchmark for future work.

Memorization. CHStone, MachSuite, and Polybench are public benchmarks and may appear in LLM training data. Memorization of original source would, however, raise repair rates roughly uniformly, whereas our results show strong variation by suite and bug type; for example, SHFT remains hard even in simpler suites. This pattern is consistent with repair reasoning that scales with design context rather than retrieval of memorized fixes.

Reproducibility and implications for repair tools. The benchmark is organized so that future methods can be evaluated without reimplementing the validation logic: each instance provides a correct version, a buggy version, the target file, and an executable oracle, and a repair system need only emit a replacement file for the script to rebuild, run the oracle, and record the outcome. This separation lets LLM-only, symbolic, localization-guided, and hybrid APR methods be compared under one executable criterion. It also points to where effort is best spent: compact kernels are near-saturated, while CHStone-like designs will need bug localization to reduce context dilution, execution feedback to reject plausible-but-wrong patches, and stronger oracles to expose defects hidden by single-vector tests. The benchmark thus identifies not only which instances are hard but which capabilities future APR systems must add beyond zero-shot generation.

Threats to Validity. *Internal validity:* the primary threat is oracle sensitivity. Bugs in code paths not exercised by single-vector tests are invisible to the oracle; the 75 rejected candidate

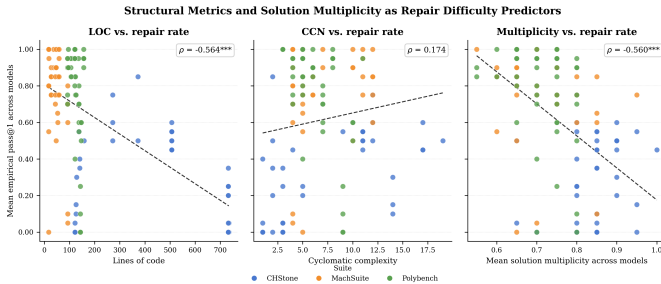


Fig. 5. Comparison of repair difficulty predictors. LOC and mean solution multiplicity are both strongly negatively correlated with mean empirical pass@1, while cyclomatic complexity is not a significant negative predictor.

instances were predominantly oracle-invisible for this reason. *External validity*: CHStone, MachSuite, and Polybench are widely cited public benchmarks that may appear in LLM training data, introducing a potential memorization confound. However, memorization would raise repair rates uniformly, whereas our results show strong structured variation by suite and bug type. We evaluate C-level functional repair only; whether patches remain correct after HLS synthesis is a downstream question beyond the scope of this benchmark. *Construct validity*: we evaluate zero-shot repair exclusively. Localization-guided or iterative repair could substantially improve results on hard instances, particularly CHStone; the benchmark harness is designed to support such evaluations directly.

Future Work. Three directions follow directly from this work. First, the benchmark harness supports drop-in evaluation of localization-guided and iterative repair methods; applying ARSP [3] or LiK [4]-style fault localization to CHStone designs is a natural next step. Second, oracle strengthening—expanding single-vector CHStone tests to cover additional key-length paths—would reduce underconstraint and produce a harder, more discriminating benchmark. Third, the current suite was constructed under a strict open-source, gcc-only constraint; incorporating proprietary HLS toolchains such as Xilinx Vitis HLS or Intel oneAPI HLS would enable synthesis-aware oracles that validate not only C-level functional correctness but also synthesizability, timing closure, and resource utilization, producing richer and more realistic benchmarks for next-generation APR evaluation.

VII. CONCLUSION

We presented the first comprehensive evaluation of LLM-based HLS bug repair, built on 125 executable benchmark instances with functional oracles spanning three open-source HLS suites and eight logic bug types. Our main findings are: (1) design context and scale, rather than bug type alone, dominate repair difficulty; (2) solution multiplicity strongly predicts repair failure and provides an LLM-native measure of oracle underconstraint; and (3) frontier models (GPT-5.4, Claude Opus 4.6) largely solve simple HLS kernels but leave complex cryptographic designs substantially unrepaired. SHFT bugs remain the hardest across all models, while semantic bug classes such as buffer indexing become reliably repairable only at frontier scale.

These results have direct implications for benchmark design: a meaningful HLS repair benchmark must sample across design complexity, not only bug type. They also motivate oracle-guided repair strategies (fault localization, iterative feedback) as the next step for improving repair on hard instances where zero-shot LLMs consistently fail to converge.

REFERENCES

- [1] H. Ahmad, J. Hooker, and W. Weimer, “CirFix: Automatically repairing defects in hardware design code,” in *Proc. 27th ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022.
- [2] K. Laeuffer, B. Fajardo, A. Ahuja, V. Iyer, B. Nikolic, and K. Sen, “RTL-Repair: Fast symbolic repair of hardware design code,” in *Proc. 29th ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [3] B. Yao, N. Wang, X. Liu *et al.*, “ARSP: Automated repair of Verilog designs via semantic partitioning,” *arXiv preprint arXiv:2508.16517*, 2025.
- [4] Y. Zhao, Z. Jiang *et al.*, “Location is key: Leveraging large language models for functional bug localization in Verilog,” *arXiv preprint arXiv:2409.15186*, 2024.
- [5] C. Xiong, C. Liu, H. Li, and X. Li, “HLSPilot: LLM-based high-level synthesis,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9.
- [6] X. Tong *et al.*, “HLSDebugger: Identification and correction of logic bugs in HLS code with LLM solutions,” *arXiv preprint arXiv:2507.21485*, 2025, iEEE Xplore document 11240664.
- [7] Z. Liu, D. Yang, X. Mao, J. He, G. Zhang, Y. Lei, and J. Wu, “SRepair: Symbolic regression-based repair for hardware design code,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.
- [8] M. Mastora and D. Sullivan, “Dictionary-guided mutation operators for automated hdl repair,” in *Proceedings of the 35th IEEE Microelectronics Design and Test Symposium (IEEE MDTs 2026)*, May 2026, to appear in IEEE Xplore.
- [9] Y.-D. Tsai, M. Liu, and H. Ren, “RTLFixer: Automatically fixing RTL syntax errors with large language models,” in *Proc. 61st ACM/IEEE Design Automation Conference (DAC)*, 2024.
- [10] Y. Ding *et al.*, “HDLdebugger: Streamlining HDL debugging with large language models,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 6, 2025.
- [11] M. Liu *et al.*, “CraftRTL: High-quality synthetic data generation for Verilog code models with correct-by-construction non-textual representations and targeted code repair,” in *Proc. Int. Conf. Computer-Aided Design (ICCAD)*, 2024.
- [12] S. Liew *et al.*, “VeriRAG: A retrieval-augmented framework for automated RTL testability repair,” *arXiv preprint arXiv:2507.15664*, 2025.
- [13] F. Cui *et al.*, “HWE-Bench: Benchmarking LLM agents on real-world hardware bug repair tasks,” *arXiv preprint arXiv:2604.14709*, 2026.
- [14] Y. Wan *et al.*, “An iteratively-refined dataset for high-level synthesis functional verification through LLM-aided bug injection,” in *Proc. IEEE/ACM Int. Conf. on LLM Aided Design (LAD)*, 2024.
- [15] S. A. Sheikholeslam and A. Ivanov, “Synthai: A multi agent generative ai framework for automated modular hls design generation,” *arXiv preprint arXiv:2405.16072*, 2024.
- [16] K. Xu *et al.*, “Automatic C/C++ repair for high-level synthesis via large language models,” *arXiv preprint*, 2023.
- [17] H. Ye *et al.*, “Software/hardware co-design for LLM and its application on HLS functional verification,” in *Proc. Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024.
- [18] Y. Hara, H. Tomiyama, S. Honda, and H. Takada, “Proposal and quantitative analysis of the CHStone benchmark program suite for practical C-based high-level synthesis,” in *Journal of Information Processing*, vol. 17, 2009.
- [19] B. Reagen, R. Adolf, Y. S. Shao, G.-Y. Wei, and D. Brooks, “MachSuite: Benchmarks for accelerator design and customized architectures,” in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2014.
- [20] L.-N. Pouchet and T. Grosser, “PolyBench: The polyhedral benchmark suite,” 2012.
- [21] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [22] T. J. McCabe, “A complexity measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [23] A. Arcuri and L. Briand, “A practical guide for using statistical tests to assess randomized algorithms in software engineering,” in *Proc. 33rd Int. Conf. Software Engineering (ICSE)*, 2011, pp. 1–10.
- [24] C. Xie, Y. Shi, X. Gu, and B. Shen, “Rethinking code complexity through the lens of large language models,” *arXiv preprint arXiv:2602.07882*, 2026.
- [25] Z. Chen *et al.*, “Automated bug fixing via LLM: A study of bug patterns and repair strategies,” *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2025.

- [26] S. Ouyang *et al.*, “Benchmarking automated program repair: An extensive study on both real-world and artificial bugs,” in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2024.